

Relaytic-AML: A Local-First Evaluation Lab for Financial-Crime Machine Learning

Author Name
Affiliation
`contact@example.com`

June 2026

Abstract

Financial-crime machine learning is usually judged by detector scores, but operational anti-money laundering (AML) work depends on evidence a team can trust: data posture, temporal validity, graph provenance, review capacity, and claim discipline. Relaytic-AML is a local-first evaluation lab for that setting. It is built as a set of cooperating specialist agents that share a governed local artifact graph: a guide for orientation, a scout for source and leakage checks, a strategist for task contracts, a scientist for challenge, a builder for bounded execution, trace reviewers for auditability, and release governors for public claims.

Relaytic began as a general local inference-engineering system. In this work it is sharpened into an AML-focused environment where a human or external agent can ask what is known, which artifacts matter, and what should happen next. The public evidence includes supporting PaySim synthetic temporal-fraud precision-recall area under the curve (PR-AUC) 0.638773, supporting Elliptic temporal graph-feature PR-AUC 0.668756, and Elliptic2 modern-context PR-AUC 0.94324 $\pm$ 0.000882, below the recorded RevClassifyDS reference of 0.974. The main contribution is a reproducible architecture for keeping experiments, review assumptions, limitations, and reported claims aligned while private data remains local by default.

1 Introduction

Anti-money-laundering systems are operational decision systems. A useful model does not only rank transactions or entities. It has to respect time, explain why a case reached a review queue, preserve graph or entity provenance, and make clear what a result does not prove. This is especially hard in financial-crime work because the most realistic data is private, licensed, regulated, or operationally sensitive.

That privacy boundary creates a scientific problem as much as an engineering problem. Public AML research often has to work with synthetic data, anonymized transaction graphs, or partial blockchain views, while real production teams care about delayed labels, changing behavior, investigation budgets, typologies, and model-risk review. A benchmark row is therefore incomplete unless the reader can see the data contract, the split rule, the operating point, and the reason a stronger interpretation is or is not allowed.

Relaytic was first built as a broader local inference lab for structured data. The AML edition is a deliberate narrowing of that idea. Instead of trying to be a generic benchmark runner, Relaytic-AML asks a more practical question: can a local machine or local server become a trustworthy evaluation lab where data, modeling work, agent assistance, review assumptions, and public claims stay connected?

The answer explored here is architectural. Relaytic-AML treats the local workspace as the authority. Models, tables, context packs, figures, and paper text are downstream of artifacts on disk. Agents are useful, but they act through bounded roles. A guide explains the current state. A scout checks data posture and leakage risk. A scientist challenges the modeling plan. A builder runs the experiment. A claim governor decides whether a result can be described publicly. The user is never expected to know which file to open first because Relaytic itself can explain the run state and export a compact context pack for another large language model (LLM) or coding agent.

The benchmarks in this paper are deliberately modest in their role. They are stress tests for the environment. A score is valuable only if the reader can see the dataset boundary, split rule, budget, operating point, limitation, and claim boundary that produced it. This paper therefore presents Relaytic-AML as an evaluation lab first and a benchmark package second.

2 Contribution and Scope

This paper is a systems paper about local-first AML evaluation. Its first contribution is the Relaytic-AML architecture: a workspace-centered lab where artifacts, not chat transcripts, carry the truth of an experiment. The second contribution is a concrete multi-agent runtime that separates guidance, scouting, scientific challenge, model building, trace review, and claim review into bounded loops. The third contribution is an evidence-cell contract that makes every reported number traceable to a dataset, split, command, artifact field, budget, operating point, and claim state. The fourth contribution is a release harness that turns local artifacts into tables, figures, source packages, and manuscript claims without letting unsupported interpretations slip through. The fifth contribution is a reproducible handoff surface: a human, local model, or external agent can ask what is known, which artifacts matter, and what safe action comes next.

The scope is deliberate. Relaytic-AML is presented as an evaluation environment and evidence system. Its current public benchmark rows support that systems claim; they are not offered as broad deployment superiority, graph-neural dominance, or equivalence to RevClassify. That boundary is part of the design rather than an afterthought.

Paper contribution	Relaytic mechanism	Evidence in this release
Local state is auditable	Run directories, manifests, traces, metric cells, tables, and release reports live in the workspace	Demonstrated by the generated paper and source package
Agent help is legible	Guide, scout, scientist, strategist, builder, trace reviewer, and claim governor roles have separate jobs and artifacts	Role boundaries and artifacts are documented in the paper and repository
Scores are traceable	Evidence cells bind every number to dataset, split, command, artifact field, budget, and claim state	Supporting PaySim, Elliptic, and Elliptic2-context rows
External handoff is governed	Redacted context packs, JSON command surfaces, skills, and Model Context Protocol adapters expose state without exposing private rows by default	OpenClaw, Claude/Codex project-skill, CLI, and Model Context Protocol use cases share the same artifact graph
Public claims stay evidence-bounded	Metric cells, limitation records, and release audits prevent unsupported promotion	Tables, figures, and source package are regenerated from the governed evidence layer

3 Problem Setting and Design Thesis

The central problem is legibility under constraint. A trained reader can understand PR-AUC, temporal splits, graph features, and review budgets, but that reader cannot know the state of a local experiment unless the system makes it legible. Relaytic-AML is built around the view that an evaluation environment should expose its state in the same way a model exposes its metrics: through stable artifacts, not through informal memory.

The design thesis is that local-first evidence can make agent-assisted AML research more reliable if three invariants hold. First, the local artifact graph must remain the authority for data posture, split decisions, metric values, model artifacts, and release state. Second, agent assistance must be role-scoped: a guide may explain, a scout may inspect, a scientist may challenge, a builder may execute, and a release governor may block a claim, but none of them should silently overwrite the evidence record. Third, every public interpretation of a result must be bounded by what the evidence cell can actually support.

This paper evaluates whether the current Relaytic-AML implementation makes those invariants concrete. The benchmark rows are useful because they exercise the architecture, not because they settle AML detection. The more important question is whether a human, a local model, or an external coding agent can enter the workspace, understand what is known, see what remains blocked, and choose a safe next action without private rows leaving the local boundary.

4 Local-First Agent Architecture

Relaytic is designed around one control rule: the user’s workspace is the authority. Raw and licensed data stay local by default. Run directories, manifests, traces, metric cells, model files, and paper assets form the durable record. Semantic caches, memory indexes, and LLM summaries are derived views. This is different from a remote-first agent that sends private rows to a hosted planner and later reconstructs provenance from a conversation.

The system is agentic, but the roles are concrete. They are small jobs with bounded permissions and visible outputs. A scout inspects source posture and split risk. A strategist turns the investigation into an executable task contract. A scientist challenges baselines and ablations. A builder executes a controlled run. A trace reviewer reconstructs decisions, branches, and claim packets. An evidence reviewer can reject an interpretation even when the model score looks attractive. A guide explains the current state to a human or exports a redacted context pack to another model.

This makes Relaytic host-neutral. OpenClaw can consume the checked-in Relaytic skill notes. Claude Code can use the project-local agent and Model Context Protocol (MCP) configuration. Codex-style skill environments can use the same skill contract. Any host can also use the command-line interface (CLI) JSON surfaces. The important privacy rule is the same across all of them: external agents should receive artifact references, aggregate metadata, commands, and redacted context packs by default, while raw rows and licensed files stay inside the governed workspace unless the operator explicitly changes policy.

Role	What it owns	Local-first boundary	Main artifacts
Operator and mandate owner	Sets goals, constraints, privacy posture, and stop/continue preferences.	Can keep all data on a controlled local machine, server, or cluster.	Mandate, policy, permission, and next-action artifacts.
Guide and assist layer	Answers where the run is, what artifacts matter, and which action is safe next.	Uses local artifacts first. Optional LLM help is advisory and redacted by default.	Guide payloads, assist turns, status, and context packs.
Scout and task-contract agents	Inspect source posture, target semantics, split validity, and leakage risk.	Work from staged local snapshots rather than mutating the original data source.	Dataset registry, source manifests, split contracts, and task reports.
Scientist and challenger agents	Propose baselines, ablations, shadow candidates, and failure explanations.	Candidate work is bounded by explicit budgets and local artifact permissions.	Experiment registry, scorecards, ablations, and shadow-trial reports.
Builder and search controller	Execute reproducible model/search plans and select thresholds on validation evidence.	Optional adapters are versioned and never become hidden sources of truth.	Run directories, model artifacts, search traces, and operating-point records.
Evidence and release governors	Decide how evidence may be described and which claims stay blocked.	Fail closed on leakage, missing provenance, unsupported interpretation, or dirty release state.	Metric cells, claim boundaries, release notes, and source-bundle audits.
External agents or LLMs	Consume exported context, propose repairs, or continue work through stable surfaces.	Receive rowless/redacted context unless policy explicitly grants richer access.	External context packs, handoff reports, and reproducible commands.

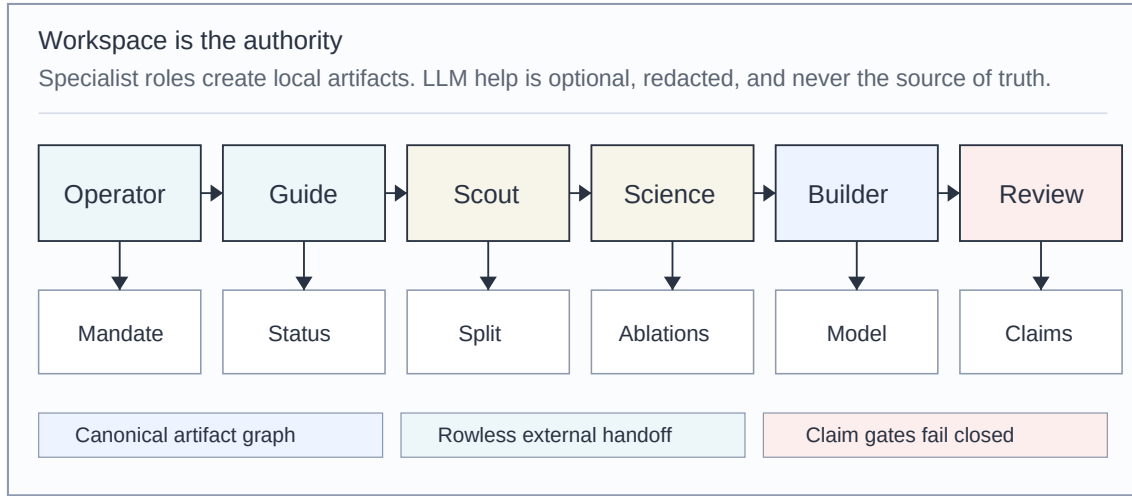
Two design choices carry most of the system. First, important work produces a local artifact that another human or agent can inspect. Second, optional intelligence is subordinate to the artifact graph. A local LLM may help phrase guidance, and a frontier model may suggest repairs, but neither becomes the source of truth unless its proposal is converted into a reproducible local artifact.

4.1 Agent Runtime, Loops, and Harness

Relaytic’s technical core is an artifact-first agent loop. Each specialist observes the local run state, reads the contracts it is allowed to read, decides a bounded next action, executes deterministic or advisory logic, writes a typed artifact, and records enough trace information for another process to audit the step. The loop is intentionally less glamorous than an open-ended chat agent. It is engineered so that progress survives process restarts, model changes, and external review.

The scout loop is deterministic-first. It converts ingestion metadata, quality checks, stationarity heuristics, target-risk signals, and column-name risk scoring into an investigation record. The strategist loop then converts that investigation state into a builder handoff: task type, target

Local-first Relaytic agent architecture



Benchmarks exercise this architecture; they do not replace the local-first artifact and claim-control thesis.

Figure 1: Local-first Relaytic agent architecture

field, split route, metric family, candidate steps, and unresolved assumptions. When an advisory model is available, its output is treated as a recommendation attached to the same artifact trail. It does not replace the deterministic contract.

The modeling harness follows the same pattern. Dataset registry artifacts describe what source is being used and what cannot be claimed from it. Split contracts define the temporal or graph partition before model selection. Candidate runners write search traces, validation decisions, calibration records, selected operating points, and test metrics. The release layer reads those outputs as evidence cells. It does not trust a score unless the score can be tied back to the split, command, budget, artifact field, and claim boundary that produced it.

Runtime surface	What the implementation builds	Failure mode it controls
Investigation loop	Source manifests, quality checks, target-risk notes, leakage warnings, and task-contract evidence	Starting model work before the task is well defined
Planning loop	Builder handoffs with task type, route, metric family, candidate steps, and open assumptions	Vague objectives, wrong metrics, or hidden modeling choices
Model/search harness	Baseline and competitive candidate runs, budget tiers, calibration, threshold selection, and operating-point artifacts	Anecdotal model development or test-set tuning
Trace and adjudication loop	Runtime spans, specialist traces, branch graphs, claim packets, replay reports, and scorecards	Uninspectable agent decisions and unsupported branch choices
Guide and assist loop	Run-state summaries, action menus, artifact shortlists, status answers, and redacted context packs	Humans or external agents getting lost in the workspace
Release harness	Evidence cells, paper tables, figures, source bundles, citations, and claim-gate reports	Release claims exceeding the evidence record

This is the part of Relaytic that matters technically. The agents are not only prompt roles. They are state machines around a governed artifact graph. The architecture favors typed files, schemas, replayable commands, and conservative gates because those are the objects a reviewer, engineer, or future agent can actually inspect. In AML, that matters more than a fluent explanation. A fluent explanation is useful only after the evidence state is already coherent.

4.2 Execution Contract and System Evidence

A full Relaytic run follows a fixed execution contract. The mandate records the user’s goal, constraints, and data-movement posture. Intake and scouting materialize source posture, task semantics, candidate target fields, leakage warnings, and split recommendations. Planning converts those findings into a builder handoff. Execution writes candidate results and operating-point artifacts. Trace review records decisions, branches, tool use, and claim packets. The release harness then decides what can appear in a table, figure, paper sentence, or external handoff.

The practical effect is that an agent cannot simply assert that a result is valid. It has to leave behind a path that other surfaces can read. A guide response, an assist turn, a mission-control view, a paper table, and a redacted context pack all read from the same artifact graph. This is why Relaytic is local-first in more than a privacy sense: the local state is also the arbitration layer for meaning.

Run stage	Primary agent surface	Artifact contract	Evidence control
Mandate and policy	Operator plus guide	Goal, data boundary, allowed actions, stop/continue preference	Defines the authorized work envelope before analysis starts
Source and task scouting	Scout	Source manifest, quality checks, target-risk notes, split-risk notes	Freezes data posture and task validity before model search
Planning handoff	Strategist	Task profile, metric family, route, candidate steps, unresolved assumptions	Converts investigation state into an executable builder contract
Search and execution	Builder/search controller	Candidate traces, validation decisions, calibration, thresholds, selected run	Separates validation-selected development from fixed test reporting
Trace and adjudication	Trace reviewer	Runtime spans, branch graph, claim packets, replay report, scorecard	Makes agent choices replayable as artifacts rather than remembered prose
Release and handoff	Evidence governor plus guide	Evidence cells, claim boundaries, paper tables, context pack	Promotes only interpretations supported by the local evidence record

Algorithm 1 shows the loop in implementation terms. The important feature is not that every specialist uses the same model, but that every specialist is forced through the same artifact, gate, and trace discipline.

Algorithm 1: Artifact-first specialist loop

```

input: run_dir, specialist_contract, policy
repeat until stop_condition(run_dir, policy):
  state <- read_allowed_artifacts(run_dir, specialist_contract)
  obs <- inspect(state)
  proposal <- deterministic_step(obs)

  if advisory_model_enabled(policy):
    note <- advisory_note(obs, redacted_context(state))
    proposal <- attach_note(proposal, note)

  decision <- gate(proposal, policy, budget, claim_contract)
  if decision.accept:
    artifact <- write_typed_artifact(run_dir, decision.payload)
    trace(role, inputs=state.refs, outputs=artifact.refs)
  else:
    blocker <- write_blocker(run_dir, decision.reason)
    trace(role, inputs=state.refs, outputs=blocker.refs)

```


`refresh_run_summary(run_dir)`

A simplified evidence cell shows the same contract at metric level:

Evidence-cell field	Example from the PaySim row	Why a reviewer needs it
Dataset identity	<code>paysim_temporal_transaction_fraud</code>	Separates synthetic proxy evidence from real-bank evidence
Split contract	Chronological step split	Prevents random-split leakage from masquerading as temporal validity
Metric and value	Test PR-AUC 0.638773	Names the exact reported quantity
Budget tier	Competitive	Shows that stronger search was used rather than a smoke run
Leakage posture	Prior-step destination history only	Explains why engineered history features are allowed
Claim state	Supporting only	Prevents the result from being promoted beyond its dataset boundary
Source artifacts	Benchmark manifest, budget contract, and claim gate	Gives the reader concrete files to inspect or regenerate

This schema-like record is the difference between a result and an anecdote. A serious reader can challenge the dataset, the split, the metric, the budget, the leakage posture, or the claim state without needing to reconstruct the run from memory.

5 Current Frontier Context

The current AML frontier is not a single leaderboard. It is a set of pressure points: real graph scale, realistic entity behavior, temporal drift, operational throughput, and reliable agent-assisted research. Relaytic-AML is designed as infrastructure around those pressure points, not as a replacement for detector papers.

Recent agentic machine-learning work also treats external state, benchmark environments, and research validity as first-class objects. SkillOpt frames agent skills as trainable external state with validation-gated edits Yang et al. [2026]. MLR-Bench evaluates whether agents can produce valid open-ended machine-learning research rather than only fluent reports Chen et al. [2025]. PaperBench studies whether agents can reproduce full machine-learning papers from scratch and reports that the best tested agent remained far below expert human replication performance Starace et al. [2025]. RE-Bench evaluates frontier research-engineering agents against human experts in realistic open-ended environments Wijk et al. [2025]. Relaytic-AML follows the same broader movement toward executable research state, but applies it to AML-specific data custody, review queues, and release claims.

PaySim is a synthetic mobile-money simulator designed to address the scarcity of legitimate public mobile-transaction datasets for fraud research Lopez-Rojas et al. [2016]. It is useful

for temporal transaction-fraud workflow evaluation, but synthetic evidence cannot by itself establish hard real-bank AML performance.

The Elliptic Bitcoin dataset introduced a public transaction graph with more than 200K transaction nodes, 234K directed payment-flow edges, and 166 node features across 49 time steps Weber et al. [2019]. That work also showed why graph evidence must be compared against strong simpler baselines rather than assumed superior.

Elliptic2 shifts the public AML benchmark center toward subgraph learning, with 121,810 labeled subgraphs inside a background graph of roughly 49M node clusters and 196M edge transactions Bellei et al. [2024]. RevTrack and RevClassify further argue that sender and receiver context around a subgraph can be a powerful and scalable signal Song et al. [2024]. These works motivate Relaytic-AML’s modern-context and limitation track, but they do not make the current Relaytic Elliptic2 row a performance contribution.

Recent AML graph work raises the bar beyond the current Relaytic evidence rows. TransX-ion frames benchmark realism around profile-aware simulation, richer entity attributes, non-template illicit synthesis, and out-of-character behavior Chen et al. [2026]. LineMVGNN and ExSTraQt focus on directed money flow, edge-aware graph views, and quasi-temporal transaction representations Poon et al. [2026] Tariq and Hassani [2026]. BlazingAML treats throughput and multi-stage graph mining as a systems problem Ye et al. [2026]. Continual graph-learning reviews emphasize drift, adaptation, class imbalance, and changing laundering behavior Deprez et al. [2025]. These papers point to a frontier where realism, scale, graph structure, time, and operations are inseparable. Relaytic-AML is complementary to those efforts: it does not claim detector parity with them, but tries to make dataset posture, split validity, budget, limitations, and release claims auditable.

The paper also follows broader machine-learning documentation and reproducibility practice. Datasheets for Datasets and Model Cards argue for explicit dataset and model reporting Gebru et al. [2021] Mitchell et al. [2019]. The NeurIPS reproducibility program highlights the need for code, data, and checklist discipline in machine-learning research Pineau et al. [2021]. Recent work on machine-learning research agents warns that coherent papers can still contain invalidated experiments, which reinforces the need for executable artifacts, reproducible commands, and explicit claim boundaries Chen et al. [2025] Starace et al. [2025].

6 Methodology: Evidence Cells, Gates, and Budgets

Relaytic-AML treats a metric as an evidence cell, not as a free-standing score. An evidence cell records the dataset identity, split contract, execution command, run or artifact reference, metric value, budget tier, leakage posture, operating-point rule, claim state, and limitation notes. A reported row is accepted only when those fields are present and internally consistent.

This creates a small theory of evidence for the system. A metric becomes scientific evidence only after it is attached to provenance, a comparison budget, and an interpretation boundary. Provenance answers where the number came from. Budget answers how much modeling effort was spent before reporting it. The interpretation boundary answers what the number is allowed to mean in public. Removing any one of those pieces turns the row back into an anecdote.

The claim gate is intentionally conservative. It can mark a row as supporting evidence, modern context, baseline-only evidence, or blocked evidence. In the current public version, no row is headline-eligible. This is not a weakness of the environment. It is the point of the environment. A strong-looking number is only useful if the system can also say what the number is allowed to mean.

The budget ladder separates quick engineering checks from serious evidence. Smoke runs test that commands and artifacts exist. Baseline runs establish conservative rows. Competitive runs add stronger feature families, validation-only threshold selection, calibration, and model search. Frozen reporting runs preserve the transformation from experiment to table, figure, and public claim. This prevents a weak first pass from being quietly promoted into a strong paper claim and also prevents a successful search from hiding how much effort was spent.

For an external reader, the important idea is simple: every number has a trail, and every claim has a boundary. The trail helps another person reproduce or challenge the result. The boundary says whether the result is a benchmark observation, an operational estimate, a limitation, or a claim that should not be made yet.

7 Evidence Operating Layer

The local-first architecture becomes concrete through an evidence operating layer. Relaytic-AML is not only a model runner. It coordinates source posture, task semantics, split discipline, model search, decision thresholds, review queues, artifacts, trace review, and claim boundaries.

Layer	What Relaytic records	Why it matters
Source and task contracts	Dataset access, target semantics, benchmark posture, and split rules	A reader can see whether the task is valid before judging the score
Execution and search	Candidate families, budgets, calibration, thresholds, selected runs, and search traces	Model development becomes inspectable instead of anecdotal
AML domain layer	Entity graphs, typology posture, review queues, delayed labels, and case evidence	The model is tied to the analyst workflow it is supposed to support
Evidence ledger	Metric cells, tables, figures, limitations, and commands	Numbers are connected to the artifacts that produced them
Claim boundaries	Allowed claims, blocked claims, and future unlock conditions	The same result cannot quietly become a stronger interpretation
Trace and replay	Runtime spans, branch graphs, tool logs, claim packets, and adjudication scorecards	Agentic decisions can be reviewed rather than trusted from prose
Handoff surfaces	Guide, status, assist, mission control, and redacted context export	Humans and external agents can continue work without guessing hidden state

This operating layer is useful even when a result cannot support a stronger interpretation. The row is not thrown away. It becomes a structured research state with a reason, an artifact reference, and a repair path.

8 What Relaytic-AML Is For

The intended user is a person or team that has data, a risky modeling question, and a need to know whether the evidence is strong enough to act on. That includes a bank team comparing a new model against an incumbent queue, a fraud group exploring a new dataset, a researcher testing a benchmark protocol, or an external agent trying to continue a run without seeing private rows.

Capability	Operational role
Local-first privacy posture	Private or licensed data can stay outside the public repo while hashes, access posture, and evidence artifacts remain inspectable
Artifact-first reproducibility	Tables and figures are tied to local JSON and model artifacts rather than loose notes
Role-scoped agent help	A guide, scout, scientist, builder, and claim reviewer have different responsibilities
Budget-aware modeling	Quick checks, baseline evidence, competitive runs, and frozen reporting are kept separate
Operational evaluation	Review-budget precision, recall, false-positive burden, and case-packet completeness sit next to model metrics
External context export	Another LLM or coding agent can receive redacted state, artifact references, and reproducible commands

For other agents, Relaytic behaves like a local evidence service rather than a private-data proxy. An OpenClaw workflow can read the Relaytic skill and call the same command surfaces. A Claude Code session can use the project-local agent notes and MCP configuration. A Codex or similar project-skill host can follow the checked-in skill contract. In all cases the safe handoff is artifact-first: the agent sees the run summary, selected artifact references, aggregate metrics, limitations, and reproduction commands. It does not need raw private rows to decide whether the next action is a rerun, a leakage repair, a stronger baseline, a data-acquisition step, or a claim edit.

This is why the project moved from a broad Relaytic system toward Relaytic-AML as the flagship story. The general architecture still matters, but AML gives it a sharper test. It forces the system to handle rare events, temporal splits, graph context, human review limits, privacy, and public-claim discipline at the same time.

Companies could use this kind of lab to challenge incumbent rules or models on the same review queue, evaluate whether a new dataset is worth deeper investment, audit whether a vendor comparison is fair, and prepare evidence packs for compliance or model-risk review. Engineering leaders could use it to test whether an agent-assisted machine-learning workflow is actually governable: whether the system knows its state, exposes its assumptions, records its choices, and refuses claims it has not earned. Researchers could use it to ask whether an AML result is supported, blocked, or mainly useful as a limitation.

9 Evaluation Environment

The current evaluation environment combines public benchmark evidence with local artifact discipline. Dataset registry artifacts define source and access posture. Split contracts define chronological, graph-snapshot, or subgraph partition rules. Benchmark runners produce model and operating-point artifacts. Tables and figures then read from those artifacts.

The current public evidence uses three tracks. PaySim is a synthetic temporal proxy. Elliptic is temporal graph-feature evidence. Elliptic2 is retained as modern subgraph context and limitation evidence because a faithful RevClassify reference-protocol reproduction has not yet been completed locally.

The environment follows three design rules. Local artifacts are the source of truth. Validation selects models, thresholds, and operating points before fixed test evaluation. Blocked evidence stays visible because hiding failed or incomplete tracks makes both human and agent-assisted research less scientific.

9.1 System Evaluation

The benchmark rows are not the only evaluation target. Relaytic itself has to behave like a usable research instrument. The current paper therefore treats system behavior as part of the evidence: whether a user or another agent can recover state, find the right artifacts, understand what is blocked, and continue without private rows leaving the workspace.

Evaluation dimension	Relaytic surface	Evidence emitted
State recovery	Guide, status fallback, assist turns, mission-control summaries, and run-summary artifacts	Current state, artifact shortlist, starter actions, and next-step options generated from local artifacts
External-agent handoff	Rowless/redacted context packs, OpenClaw notes, Claude/Codex skill contracts, JSON CLI, and MCP surfaces	Commands, artifact references, limitations, and claim posture without raw rows by default
Metric auditability	Evidence cells, metric-cell audit, command ledger, budget contracts, and release manifests	Each public row binds value, dataset, split, budget, leakage posture, and claim state
Agent traceability	Runtime spans, specialist traces, branch graph, claim packets, replay report, and adjudication scorecard	Agent decisions materialized as artifacts rather than only remembered in conversation
Claim governance	Public-claim lint, claim gate matrix, limitation records, and release checklist	Tables, figures, and paper text regenerated from the same governed evidence layer

The PaySim competitive row illustrates the contract. The run uses a chronological split, excludes forbidden balance fields, builds prior-step destination history, selects candidates on validation evidence, and freezes the test operating point. The resulting PR-AUC is reported because the evidence cell is complete. It is not promoted to a real-bank AML claim because the data is

synthetic. That refusal is a system result: Relaytic preserved the useful score while preventing the more attractive but unsupported sentence.

10 Benchmark Protocol

The benchmark protocol is deliberately subordinate to the architecture. Its purpose is to check whether Relaytic can produce traceable evidence, respect split and leakage contracts, expose operational assumptions, and block unsupported claims. A single score does not define the value of the system.

The public rows separate baseline and competitive evidence. Competitive rows use stronger feature families, candidate search, calibration, and validation-only operating-point selection. The point is to make modeling effort visible and to keep future reruns comparable rather than letting one attractive number become the whole story.

The evidence summary below should be read as a claim-boundary table as much as a performance table. The numbers matter, but the posture column is what stops them from being overstated.

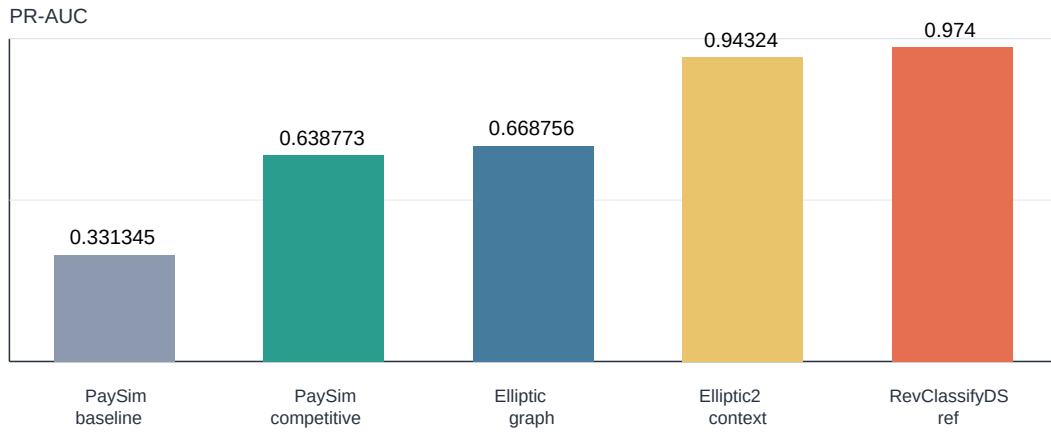
Evidence row	Metric	Value	Claim posture
PaySim baseline	test PR-AUC	0.331345	baseline-only
PaySim competitive	test PR-AUC	0.638773	supporting-only synthetic temporal proxy
PaySim competitive	precision at review budget	0.703336	supporting-only
PaySim competitive	recall at review budget	0.471584	supporting-only
Elliptic graph-feature	test PR-AUC	0.668756	supporting-only graph-feature evidence
Elliptic graph-feature	precision at review budget	1	supporting-only
Elliptic graph-feature	recall at review budget	0.056604	supporting-only
Elliptic2 context	official-partition PR-AUC mean	0.94324	modern context only
Elliptic2 context	official-partition PR-AUC std	0.000882	modern context only
RevClassifyDS reference	published PR-AUC	0.974	reference context, not parity

11 Results

The PaySim competitive row improves over the PaySim baseline inside the synthetic temporal-fraud contract. The Elliptic graph-feature row is useful supporting graph evidence, but it is not a graph-neural superiority claim. The Elliptic2 context row is strong enough to justify more work, but not enough to claim parity with the RevClassifyDS reference or to make an Elliptic2 performance contribution.

The more important result is the behavior of the environment. Relaytic can carry a useful score

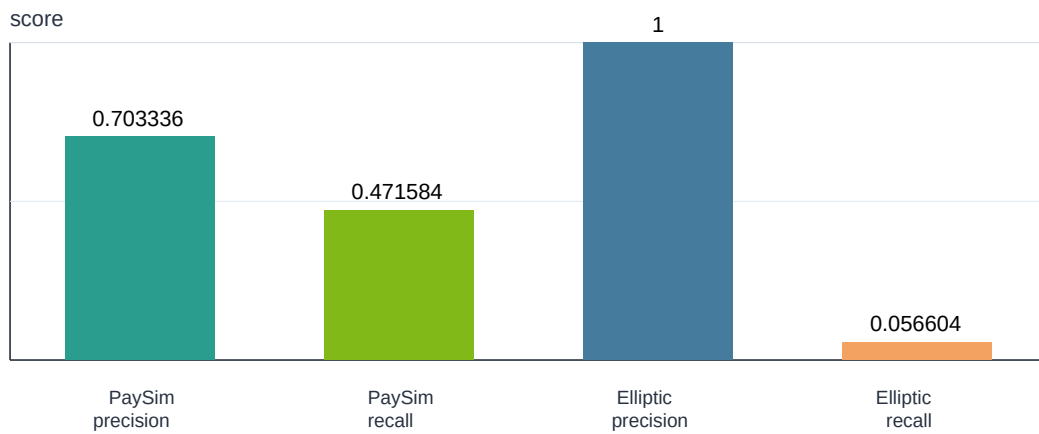
PR-AUC evidence rows



Values come from fixed local evidence cells; rows are supporting evidence, not headline claims.

Figure 2: PR-AUC evidence rows

Review-budget operating points



Values come from fixed local evidence cells; rows are supporting evidence, not headline claims.

Figure 3: Review-budget operating points

Claim boundaries and future unlocks

Every track remains useful, but stronger claims require specific evidence upgrades.

Track	Current role	Blocked claim	Future unlock	Current posture
PaySim	synthetic temporal proxy	real-bank superiority	real/partner holdout	supporting
Elliptic	graph-feature support	graph-neural superiority	repeated graph release budget	supporting
Elliptic2	modern subgraph context	RevClassify parity	faithful parity + cohort proof	context only
Operational	review-budget support	hard business value	same-queue incumbent	supporting

Figure 4: Claim boundaries and future unlocks

and still refuse the stronger sentence. In financial-crime machine learning, that refusal is not cosmetic. It is part of scientific and operational honesty.

Track	Current paper use	Blocked stronger claim	Evidence needed before promotion	Gate status
PaySim temporal proxy	supporting synthetic temporal-fraud evidence	real-bank AML superiority	partner or real holdout with frozen evaluation budget	supporting only
Elliptic graph-feature	supporting temporal graph-feature evidence	graph-neural or graph benchmark superiority	repeated graph evaluation budget against strong feature baselines	supporting only
Elliptic2 subgraph	modern subgraph context and limitation evidence	Elliptic2 performance contribution or reference-method match	faithful RevClassify reproduction or leakage-resistant subgraph protocol	context only
Operational review layer	supporting review-budget estimates	hard analyst-hour or business-value claim	complete case packets and same-queue incumbent comparison	supporting only

12 Discussion

The practical value of Relaytic-AML is not that it replaces a compliance platform or wins one benchmark table. Its value is that it gives risk, fraud, or AML teams a local evidence lab for unknown datasets, incumbent challenges, review-budget tradeoffs, leakage checks, and claim discipline. A team evaluating a new dataset can inspect whether the result is a model win, an

environment win, a proxy-only result, or a blocked claim.

For research teams and agentic machine-learning workflows, the same structure is a guard against coherent but invalid experiments. External agents can read the structured artifacts, see the claim posture, and propose the next benchmark action without inferring hidden state from prose. The strongest story is the artifact discipline: Relaytic-AML demonstrates that an ambitious evaluation system can keep progress and restraint in the same loop.

The same structure also helps when an experiment changes direction. A row can tell a team that a dataset is proxy-only, that a split needs repair, that a graph-native candidate must beat strong tabular features, that a review-queue claim needs a same-queue incumbent, or that a public sentence is stronger than the evidence permits. In high-risk financial-crime settings, those answers save time because they turn ambiguity into the next concrete engineering step.

An organization would use Relaytic-AML not as a replacement for domain expertise, but as an evaluation layer that makes difficult assumptions explicit: what data was allowed to move, what split was used, what budget was spent, whether the threshold was chosen on validation evidence, what a reviewer would see, and which interpretation the evidence can support.

13 Limitations

The current public evidence is not a deployment validation. PaySim is synthetic mobile-money evidence, so it cannot establish real-bank AML superiority. The Elliptic row is a temporal graph-feature result, not proof that a graph-neural model is better. Elliptic2 is modern context, not a Relaytic performance contribution, because faithful reference-protocol reproduction and cohort equivalence still need more work.

The operational evidence also remains early. The review-budget rows are useful for showing how Relaytic connects model output to analyst capacity, but they do not prove analyst-hour savings against an incumbent queue. A stronger version of this work should include complete case packets, a same-queue incumbent comparison, and a partner-approved or otherwise realistic holdout.

The paper therefore argues for Relaytic-AML as a useful local evaluation environment. The current benchmark rows demonstrate the architecture and evidence discipline. They do not close the broader detector question, which should be tested with stronger holdouts, larger search budgets, and partner-grade operational comparisons.

14 Future Work

The next step is to make the architecture more robust against misinterpretation and the evidence more operationally realistic. Relaytic itself should be evaluated as an object of study: whether the guide, scout, scientist, builder, and evidence reviewer agree about the same local run state; whether a first-time user can recover the right artifacts without knowing the repository; and whether an external model can use a redacted context pack to propose a useful repair without inventing unsupported claims.

The AML evidence should move in parallel. A stronger version needs a partner-approved or otherwise realistic holdout, faithful Elliptic2 reference reproduction, stronger graph-native candidates, continual-learning experiments, and same-queue business-value comparisons. The point is not to replace the current architectural story with a leaderboard story. The point is to

give the architecture harder evidence to govern.

A useful next paper would therefore have two coupled evaluations. The detector evaluation would test stronger AML candidates under frozen budgets and leakage-resistant splits. The environment evaluation would test whether humans and agents can recover state, identify missing evidence, choose the right next action, and avoid unsupported claims. That would turn Relaytic’s usability promise into a measurable benchmark rather than a narrative assertion.

The longer-term goal is still broader than AML. Relaytic should become a general local evaluation laboratory for structured, temporal, and graph machine learning. AML is the current flagship because it forces the system to handle privacy, time, graph context, human review, and claim discipline together.

15 Reproducibility

The code, paper source, figures, tables, and public evidence artifacts are in the Relaytic repository. The public repo keeps raw private or licensed data out of version control. Where a benchmark requires local data, the command ledger describes the expected local paths and access posture.

A compact Windows PowerShell reproduction path for the public paper assets is:

```
py -3.11 -m pip install -e ".[full]"
py -3.11 -m relaytic.ui.cli release-safety paper-tables --format json
py -3.11 -m relaytic.ui.cli release-safety paper-draft --format json
py -3.11 -m relaytic.ui.cli release-safety paper-release --format json
py -3.11 -m relaytic.ui.cli release-safety paper-arxiv-source --format json
py -3.11 -m relaytic.ui.cli scan-git-safety
```

The same path on macOS or Linux is:

```
python3 -m pip install -e ".[full]"
python3 -m relaytic.ui.cli release-safety paper-tables --format json
python3 -m relaytic.ui.cli release-safety paper-draft --format json
python3 -m relaytic.ui.cli release-safety paper-release --format json
python3 -m relaytic.ui.cli release-safety paper-arxiv-source --format json
python3 -m relaytic.ui.cli scan-git-safety
```

For readers, the README and this paper are the intended entry points. The README explains the current project shape: Relaytic remains the general package and command-line interface, while Relaytic-AML is the flagship AML edition used for this paper. Lower-level JSON reports, tables, figure sources, and TeX files are reproducibility machinery, not documents a first reader should have to discover manually.

16 Author Use of AI Assistance

Large-language-model tools assisted with drafting, editing, repository inspection, and consistency checks. The scientific framing, claim boundaries, experimental interpretation, and final manuscript remain the author’s work. The tools are not listed as authors.

17 Conclusion

Relaytic-AML should be read as a local-first AML evaluation-lab paper. The current work is valuable because it makes the operating idea concrete. Data stays locally governed. Specialist roles create inspectable artifacts. External agents receive structured context instead of hidden state. Claim boundaries keep manuscript claims aligned with evidence. The result is a system where model development, privacy, agent assistance, and paper claims are connected by one artifact graph.

The right test for this version is therefore whether the repository makes the current evidence easier to inspect, easier to challenge, and harder to oversell. That is the claim this paper is prepared to make.

References

- Claudio Bellei, Muhua Xu, Ross Phillips, Tom Robinson, Mark Weber, Tim Kaler, Charles E. Leiserson, Arvind, and Jie Chen. The shape of money laundering: Subgraph representation learning on the blockchain with the elliptic2 dataset, 2024. URL <https://arxiv.org/abs/2404.19109>.
- Hui Chen, Miao Xiong, Yujie Lu, Wei Han, Ailin Deng, Yufei He, Jiaying Wu, Yibo Li, Yue Liu, and Bryan Hooi. Mlr-bench: Evaluating ai agents on open-ended machine learning research, 2025. URL <https://arxiv.org/abs/2505.19955>.
- Keyang Chen, Mingxuan Jiang, Yongsheng Zhao, Zeping Li, Zaiyuan Chen, Weiqi Luo, Zhixin Li, Sen Liu, Yinan Jing, Guangnan Ye, Xihong Wu, and Hongfeng Chai. Transxion: A high-fidelity graph benchmark for realistic anti-money laundering, 2026. URL <https://arxiv.org/abs/2604.17420>.
- Bruno Deprez, Wei Wei, Wouter Verbeke, Bart Baesens, Kevin Mets, and Tim Verdonck. Advances in continual graph learning for anti-money laundering systems: A comprehensive review, 2025. URL <https://arxiv.org/abs/2503.24259>.
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daume III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723. URL <https://arxiv.org/abs/1803.09010>.
- Edgar Alonso Lopez-Rojas, Ahmad Elmir, and Stefan Axelsson. Paysim: A financial mobile money simulator for fraud detection. In *Proceedings of the 28th European Modeling and Simulation Symposium*, 2016. URL https://www.msc-les.org/proceedings/emss/2016/E_MSS2016_249.pdf.
- Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019. doi: 10.1145/3287560.3287596. URL <https://arxiv.org/abs/1810.03993>.
- Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d’Alche Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1–20, 2021. URL <https://www.jmlr.org/papers/v22/20-303.html>.

- Chung-Hoo Poon, James Kwok, Calvin Chow, and Jang-Hyeon Choi. Linemvgnn: Anti-money laundering with line-graph-assisted multi-view graph neural networks, 2026. URL <https://arxiv.org/abs/2603.23584>.
- Kiwhan Song, Mohamed Ali Dhraief, Muhua Xu, Locke Cai, Xuhao Chen, Arvind, and Jie Chen. Identifying money laundering subgraphs on the blockchain. In *Proceedings of the 5th ACM International Conference on AI in Finance*, 2024. doi: 10.1145/3677052.3698635. URL <https://arxiv.org/abs/2410.08394>.
- Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. Paperbench: Evaluating ai’s ability to replicate ai research, 2025. URL <https://arxiv.org/abs/2504.01848>.
- Haseeb Tariq and Marwan Hassani. Extracting money laundering transactions from quasi-temporal graph representation, 2026. URL <https://arxiv.org/abs/2604.02899>.
- Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics, 2019. URL <https://arxiv.org/abs/1908.02591>.
- Hjalmar Wijk, Tao Roa Lin, Joel Becker, Sami Jawhar, Neev Parikh, T. Broadley, Lawrence Chan, Michael Chen, Joshua M. Clymer, Jai Dhyan, Elena Elicheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Jun Koba Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. Re-bench: Evaluating frontier ai r&d capabilities of language model agents against human experts. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 66772–66832, 2025. URL <https://proceedings.mlr.press/v267/wijk25a.html>.
- Yifan Yang, Ziyang Gong, Wei-quan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. Skillopt: Executive strategy for self-evolving agent skills, 2026. URL <https://arxiv.org/abs/2605.23904>.
- Haojie Ye, Arjun Laxman, Yichao Yuan, Krisztian Flautner, and Nishil Talati. Blazingaml: High-throughput anti-money laundering (aml) via multi-stage graph mining, 2026. URL <https://arxiv.org/abs/2604.12241>.